

SAA_Actividad 6.4 - ALVARO FERNANDEZ BECERRA

✓ Fundamentos de tensorflow.

Vamos a aprender las nociones básicas de tensorflow

- Introducción a tensores
- Obtener info de tensores
- Manipular tensores
- Tensores & numpy
- usar `@tf.function` (una forma de acelerar las funciones de python)
- usar `gpus`

Introducción a los Tensores

los tensores son algo así como los arrays de NumPy, para los fines de este cuaderno y de ahora en adelante, puedes pensar en un tensor como una representación numérica multidimensional (también referida como n-dimensional, donde n puede ser cualquier número) de algo. Donde algo puede ser casi cualquier cosa que puedas imaginar:

- Podría ser los propios números (usando tensores para representar el precio de las casas).
- Podría ser una imagen (usando tensores para representar los píxeles de una imagen).
- Podría ser texto (usando tensores para representar palabras).
- O podría ser alguna otra forma de información (o datos) que quieras representar con números.

La principal diferencia entre los tensores y los arrays de NumPy (también un array n-dimensional de números) es que los tensores pueden utilizarse en GPUs (unidades de procesamiento gráfico) y TPUs (unidades de procesamiento tensorial).

La ventaja de poder ejecutarse en GPUs y TPUs es la computación más rápida, esto significa que si quisiéramos encontrar patrones en las representaciones numéricas de nuestros datos, generalmente podemos encontrarlos más rápido usando GPUs y TPUs.

Lo primero que haremos es importar TensorFlow bajo el alias común `tf`.

```
# importa tensorflow e imprime su versión
import tensorflow as tf
print("Versión de TensorFlow:", tf.__version__)
```

```
Versión de TensorFlow: 2.20.0
```

```
#también importamos numpy para luego hacer alguna operación

import numpy as np
```

Creando Tensores con `tf.constant()`

Como se mencionó anteriormente, en general, normalmente no crearás tensores tú mismo. Esto se debe a que TensorFlow tiene módulos integrados (como `tf.io` y `tf.data`) que son capaces de leer tus fuentes de datos y convertirlas automáticamente en tensores y luego, más adelante, los modelos de redes neuronales procesarán estos por nosotros.

Pero por ahora, porque estamos familiarizándonos con los tensores en sí mismos y cómo manipularlos, veremos cómo podemos crearlos nosotros mismos.

Comenzaremos usando `tf.constant()`

✓ Se puede crear un tensor escalar (sin dimensión):

```
# crea un escalar con tensorflow y muestras su forma (shape)
escalar = tf.constant(42)
print("Escalar:", escalar)
print("Shape:", escalar.shape)
```

```
Escalar: tf.Tensor(42, shape=(), dtype=int32)
Shape: ()
```

```
# con el atributo ndim, muestra el número de dimensiones
print("Número de dimensiones:", escalar.ndim)
```

```
Número de dimensiones: 0
```

✓ Se puede crear un tensor vector (una dimensión):

```
# crea un vector y observa su clase (type)
```

```
vector = tf.constant([1, 2, 3])
print("Vector:", vector)
print("Tipo:", type(vector))
```

```
Vector: tf.Tensor([1 2 3], shape=(3,), dtype=int32)
Tipo: <class 'tensorflow.python.framework.ops.EagerTensor'>
```

```
print("Dimensiones del vector:", vector.ndim)
```

```
Dimensiones del vector: 1
```

✓ Se puede crear un tensor matriz (dos dimensiones):

```
# crea una matriz con tensorflow, ¿qué tipos tienen sus elementos?
matriz = tf.constant([[1, 2],
                      [3, 4],
                      [5, 6]])
print("Matriz:", matriz)
print("Dtype (tipo de elementos):", matriz.dtype)
```

```
Matriz: tf.Tensor(
[[1 2]
 [3 4]
 [5 6]], shape=(3, 2), dtype=int32)
Dtype (tipo de elementos): <dtype: 'int32'>
```

```
print("Dimensiones de la matriz:", matriz.ndim)
```

```
Dimensiones de la matriz: 2
```

Los tensores en TensorFlow pueden contener diversos tipos de datos, clasificados principalmente por su dtype (tipo de dato). Aquí se presenta una lista de los tipos de datos más comunes en TensorFlow:

- `tf.float32`: Punto flotante de 32 bits.
- `tf.float64`: Punto flotante de 64 bits, también conocido como `tf.double`.
- `tf.int8`: Entero de 8 bits.
- `tf.int16`: Entero de 16 bits.
- `tf.int32`: Entero de 32 bits.
- `tf.int64`: Entero de 64 bits, utilizado comúnmente para índices.
- `tf.uint8`: Entero sin signo de 8 bits.
- `tf.uint16`: Entero sin signo de 16 bits.
- `tf.uint32`: Entero sin signo de 32 bits.
- `tf.uint64`: Entero sin signo de 64 bits.
- `tf.bool`: Booleano.
- `tf.string`: Cadenas de texto.
- `tf.complex64`: Número complejo compuesto por dos floats de 32 bits.
- `tf.complex128`: Número complejo compuesto por dos floats de 64 bits, también conocido como `tf.double`.
- `tf.qint8`: Entero de 8 bits utilizado en cuantificación.
- `tf.qint16`: Entero de 16 bits utilizado en cuantificación.
- `tf.qint32`: Entero de 32 bits utilizado en cuantificación.
- `tf.quint8`: Entero sin signo de 8 bits utilizado en cuantificación.
- `tf.quint16`: Entero sin signo de 16 bits utilizado en cuantificación.

Estos tipos de datos permiten que TensorFlow maneje eficientemente una amplia gama de datos para diferentes tipos de operaciones, incluyendo cálculos matemáticos, manipulación de imágenes, procesamiento de texto, y mucho más.

podemos especificar el tipo de dato de los componentes del tensor, por ejemplo, float, con el parámetro dtype:

```
## crea un tensor matriz con elementos de tipo float16
matriz_float = tf.constant([[1.0, 2.0],
                           [3.0, 4.0]], dtype=tf.float16)
print("Matriz float16:", matriz_float)
print("Dtype:", matriz_float.dtype)
```

```
Matriz float16: tf.Tensor(
[[1. 2.]
 [3. 4.]], shape=(2, 2), dtype=float16)
Dtype: <dtype: 'float16'>
```

▼ Se puede crear un tensor multidimensional:

```
# crea un tensor (3 o más dimensiones)
tensor_3d = tf.constant([[[1, 2, 3],
                           [4, 5, 6]],
                         [[7, 8, 9],
                          [10, 11, 12]]])

tensor_3d

<tf.Tensor: shape=(2, 2, 3), dtype=int32, numpy=
array([[[ 1,  2,  3],
         [ 4,  5,  6]],
       [[ 7,  8,  9],
        [10, 11, 12]]], dtype=int32)>
```

```
# muestra sus dimensiones y su forma
print("Dimensiones:", tensor_3d.ndim)
print("Forma (shape):", tensor_3d.shape)
```

```
Dimensiones: 3
Forma (shape): (2, 2, 3)
```

El ejemplo anterior también se conoce como un tensor de rango 3 (3 dimensiones), sin embargo, un tensor puede tener una cantidad arbitraria (ilimitada) de dimensiones.

Por ejemplo, podrías convertir una serie de imágenes en tensores con forma (224, 224, 3, 32), donde:

- 224, 224 (las primeras 2 dimensiones) son la altura y anchura de las imágenes en píxeles.
- 3 es el número de canales de color de la imagen (rojo, verde, azul).
- 32 es el tamaño del lote (el número de imágenes que una red neuronal ve en un momento dado).

Todas las variables anteriores que hemos creado son en realidad tensores. Pero también puedes escucharlas referidas con sus diferentes nombres (los que les dimos):

- **escalar**: un solo número.
- **vector**: un número con dirección (por ejemplo, velocidad del viento con dirección).
- **matriz**: un array bidimensional de números.
- **tensor**: un array de números n-dimensional (donde n puede ser cualquier número, un tensor de 0 dimensiones es un escalar, un tensor de 1 dimensión es un vector).

Para añadir a la confusión, los términos matriz y tensor a menudo se usan indistintamente.

En adelante, dado que estamos utilizando TensorFlow, todo lo que nos referimos y usamos serán tensores.

Para más información sobre la diferencia matemática entre escalares, vectores y matrices, consulta la [publicación de álgebra visual de Math is Fun](#).

▼ crear tensores con `tf.Variable`

- https://www.tensorflow.org/api_docs/python/tf/Variable

También puedes (aunque raramente lo harás, porque cuando trabajas con datos, los tensores se crean automáticamente) crear tensores usando `tf.Variable()`.

La diferencia entre `tf.Variable()` y `tf.constant()` es que los tensores creados con `tf.constant()` son inmutables (no pueden cambiarse, solo pueden usarse para crear un nuevo tensor), mientras que, los tensores creados con `tf.Variable()` son mutables (pueden cambiarse).

Manipulando tensores `tf.Variable`

Los tensores creados con `tf.Variable()` pueden ser modificados en su lugar utilizando métodos tales como:

- `.assign()` - asigna un valor diferente a un índice particular de un tensor variable.
- `.assign_add()` - añade a un valor existente y lo reasigna en un índice particular de un tensor variable.

```
# crea un tensor mutable y uno no mutable, mostrar las diferencias
```

```
# mutable
```

```
tensor_mutable = tf.Variable([1, 2, 3])
```

```
# no mutable
```

```
tensor_constante = tf.constant([1, 2, 3])
```

```
print("Mutable (Variable):", tensor_mutable)
```

```
print("Inmutable (constant):", tensor_constante)
```

```
Mutable (Variable): <tf.Variable 'Variable:0' shape=(3,) dtype=int32, numpy=array([1, 2, 3])>
Inmutable (constant): tf.Tensor([1 2 3], shape=(3,), dtype=int32)
```

```
# cambia uno de los elementos en un mutable con el operador = -> ERROR!
```

```
try:
```

```
    tensor_mutable[0] = 99
```

```
except TypeError as e:
```

```
    print("ERROR:", e)
```

```
ERROR: 'ResourceVariable' object does not support item assignment
```

```
# utiliza assign para asignar un valor a un elemento de un tensor
```

```
tensor_mutable.assign([10, 2, 3])
```

```
print("Tras assign:", tensor_mutable)
```

```
Tras assign: <tf.Variable 'Variable:0' shape=(3,) dtype=int32, numpy=array([10, 2, 3])>
```

```
## comprueba que no se puede usar assign en un tensor constante
```

```
try:
```

```
    tensor_constante.assign([10, 2, 3])
```

```
except AttributeError as e:
```

```
    print("ERROR:", e)
```

```
ERROR: 'tensorflow.python.framework.ops.EagerTensor' object has no attribute 'assign'
```

```
# Crea una variable tensor de longitud 5 con arange de numpy;
```

```
#asigna múltiples valores con el método assign
```

```
variable_5 = tf.Variable(np.arange(5))
```

```
print("Original:", variable_5)
```

```
# Asignamos multiples valores nuevos
variable_5.assign([10, 20, 30, 40, 50])
print("Tras assign múltiple:", variable_5)
```

```
Original: <tf.Variable 'Variable:0' shape=(5,) dtype=int64, numpy=array([0, 1, 2, 3, 4])>
Tras assign múltiple: <tf.Variable 'Variable:0' shape=(5,) dtype=int64, numpy=array([10,
```

```
#añade el valor 10 a todos los elementos del tensor
```

```
variable_5.assign_add([10, 10, 10, 10, 10])
print("Tras sumar 10 a cada elemento:", variable_5)
```

```
Tras sumar 10 a cada elemento: <tf.Variable 'Variable:0' shape=(5,) dtype=int64, numpy=ar
```

✓ crear tensores aleatorios

`tf.random.uniform` y `tf.random.normal`

Los tensores aleatorios son tensores de algún tamaño arbitrario que contienen números aleatorios.

¿Por qué querías crear tensores aleatorios?

Esto es lo que las redes neuronales utilizan para inicializar sus pesos (patrones) que están intentando aprender en los datos.

Por ejemplo, el proceso de aprendizaje de una red neuronal a menudo implica tomar un array n-dimensional aleatorio de números y refinarlos hasta que representen algún tipo de patrón (una manera comprimida de representar los datos originales).

`tf.random.Generator.from_seed`

El método `tf.random.Generator.from_seed(seed)` en TensorFlow crea un generador de números aleatorios a partir de una **semilla fija** para reproducibilidad.

Ejemplo

```
import tensorflow as tf

gen = tf.random.Generator.from_seed(123)
random_numbers = gen.normal([3])
print(random_numbers.numpy())
```

```
# genera dos tensores aleatorios de shape(3,2) con la semilla 42 y comprueba si son o no
generador = tf.random.Generator.from_seed(42)
tensor_aleatorio1 = generador.normal(shape=(3, 2))

# Creamos otro generador con la misma semilla
generador2 = tf.random.Generator.from_seed(42)
tensor_aleatorio2 = generador2.normal(shape=(3, 2))

print("Tensor 1:\n", tensor_aleatorio1)
print("Tensor 2:\n", tensor_aleatorio2)
```

```
Tensor 1:
tf.Tensor(
[[-0.7565803 -0.06854702]
 [ 0.07595026 -1.2573844 ]
 [-0.23193763 -1.8107855 ]], shape=(3, 2), dtype=float32)
Tensor 2:
tf.Tensor(
[[-0.7565803 -0.06854702]
 [ 0.07595026 -1.2573844 ]
 [-0.23193763 -1.8107855 ]], shape=(3, 2), dtype=float32)
¿Son iguales? True
```

- Ambos tensores serían iguales

▼ `tf.random.shuffle`

El método `tf.random.shuffle` en TensorFlow mezcla aleatoriamente el orden de un tensor en la primera dimensión.

Ejemplo

```
import tensorflow as tf

tensor = tf.constant([1, 2, 3, 4, 5])
shuffled_tensor = tf.random.shuffle(tensor)
print(shuffled_tensor.numpy())
```

`shuffle` incorpora un parámetro `seed` para semilla *local*

```
# genera un tensor y mézclalo con el método shuffle de tensorflow
tensor_original = tf.constant([1, 2, 3, 4, 5])
tensor_mezclado = tf.random.shuffle(tensor_original)
print("Original:", tensor_original.numpy())
print("Mezclado:", tensor_mezclado.numpy())
# ejecuta varias veces y comprueba que da diferentes resultados cada vez
```

```
Original: [1 2 3 4 5]
Mezclado: [4 1 5 2 3]
```

```
# crea un tensor esta vez utilizando semilla
tensor_con_semilla = tf.constant([10, 20, 30, 40, 50])
print("Tensor:", tensor_con_semilla.numpy())
```

```
Tensor: [10 20 30 40 50]
```

```
#mezcla utilizando semilla
```

```
tensor_mezclado_semilla = tf.random.shuffle(tensor_con_semilla, seed=42)
print("Mezclado con semilla:", tensor_mezclado_semilla.numpy())
```

```
Mezclado con semilla: [20 10 30 50 40]
```

Prueba a ejecutar la celda anterior varias veces, ¿Por qué no sale siempre el mismo orden?

- ▼ Es debido a la regla #4 de la documentación de [tf.random.set_seed\(\)](#).

"4. Si se establecen tanto la semilla global como la semilla de la operación: Ambas semillas se utilizan en conjunto para determinar la secuencia aleatoria."

`tf.random.set_seed(42)` establece la semilla global, y el parámetro `seed` en `tf.random.shuffle(seed=42)` establece la semilla de la operación.

Porque, "Las operaciones que dependen de una semilla aleatoria en realidad la derivan de dos semillas: las semillas global y de nivel de operación."

```
# para mezclar siempre en el mismo orden
# establece el global random seed
tf.random.set_seed(42)

# y el random seed de la operación de mezcla
tensor_mezclado_fijo = tf.random.shuffle(tensor_con_semilla, seed=42)
print("Mezclado con semilla global + local:", tensor_mezclado_fijo.numpy())
```

Mezclado con semilla global + local: [40 10 20 30 50]

- Ahora por muchas veces que se ejecute siempre sale el mismo resultado

✓ Otras formas de crear tensores

Aunque raramente las uses (recuerda, muchas operaciones de tensores se realizan detrás de escena por ti), puedes usar `tf.ones(.)` para crear un tensor de unos y `tf.zeros(.)` para crear un tensor de ceros.

```
# crea un tensor con la función ones de forma (3,2)
tensor_unos = tf.ones(shape=(3, 2))
print(tensor_unos)
```

```
tf.Tensor(
[[1. 1.]
 [1. 1.]
 [1. 1.]], shape=(3, 2), dtype=float32)
```

```
# crea un tensor con la función zeros de forma (3,2)
tensor_ceros = tf.zeros(shape=(3, 2))
print(tensor_ceros)
```

```
tf.Tensor(
[[0. 0.]
 [0. 0.]
 [0. 0.]], shape=(3, 2), dtype=float32)
```

También puedes convertir arrays de NumPy en tensores.

Recuerda, la principal diferencia entre los tensores y los arrays de NumPy es que los tensores pueden ejecutarse en GPUs.

🔑 Nota: Una matriz o tensor típicamente se representa por una letra mayúscula (por ejemplo, X o A) mientras que un vector se representa típicamente por una letra minúscula (por ejemplo, y o b).


```
# crea un array de numpy con la función arange para dar valores del 1 al 24 con dtype=np.int32

array_numpy = np.arange(1, 25, dtype=np.int32)
print("Array numpy:", array_numpy)

# crea un tensor a partir del array anterior, dándole la forma (2,4,3) (en total 24 elementos)

tensor_desde_numpy = tf.constant(array_numpy, shape=(2, 4, 3))
print("Tensor con shape (2,4,3):\n", tensor_desde_numpy)
```

Array numpy: [1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]

Tensor con shape (2,4,3):

```
tf.Tensor(
[[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

 [[13 14 15]
 [16 17 18]
 [19 20 21]
 [22 23 24]]], shape=(2, 4, 3), dtype=int32)
```

✓ Obteniendo información de los tensores (forma, rango, tamaño)

Habrás momentos en los que querrás obtener diferentes piezas de información de tus tensores, en particular, debes conocer el siguiente vocabulario sobre tensores:

- **Forma (shape):** La longitud (número de elementos) de cada una de las dimensiones de un tensor.
- **Rango (rank):** El número de dimensiones de un tensor. Un escalar tiene rango 0, un vector tiene rango 1, una matriz tiene rango 2, un tensor tiene rango n.
- **Eje o Dimensión (axis):** Una dimensión particular de un tensor.
- **Tamaño (size):** El número total de elementos en el tensor.

Especialmente utilizarás estos cuando estés intentando alinear las formas de tus datos con las formas de tu modelo. Por ejemplo, asegurándote de que la forma de tus tensores de imagen sea la misma que la forma de la capa de entrada de tu modelo.

```
# Con este tensor de rango 4 y explora su forma, rango y longitud
rank_4_tensor = tf.zeros([2, 3, 4, 5])
rank_4_tensor
```

```
<tf.Tensor: shape=(2, 3, 4, 5), dtype=float32, numpy=
array([[[[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]],

       [[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]],

       [[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]]]], dtype=float32)
```

```
[[[0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.]],

 [[0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.]],

 [[0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.],
  [0., 0., 0., 0., 0.]]], dtype=float32)>
```

```
# muestra a continuación:
# los tipos de datos de cada elemento
# número de elementos del eje 0
# número de elementos del último eje
# número total de elementos
print("Tipo de datos:", rank_4_tensor.dtype)
print("Elementos en el eje 0:", rank_4_tensor.shape[0])
print("Elementos en el ultimo eje:", rank_4_tensor.shape[-1])
print("Total de elementos:", tf.size(rank_4_tensor).numpy())
```

```
Tipo de datos: <dtype: 'float32'>
Elementos en el eje 0: 2
Elementos en el ultimo eje: 5
Total de elementos: 120
```

▼ Se pueden indexar tensores como en numpy

```
# muestra los dos primeros elementos de cada dimensión de rank_4_tensor

print(rank_4_tensor[:, :2, :2, :2])
```

```
tf.Tensor(
[[[0. 0.]
  [0. 0.]]

 [[0. 0.]
  [0. 0.]]]

 [[[0. 0.]
  [0. 0.]]

 [[0. 0.]
  [0. 0.]]]], shape=(2, 2, 2, 2), dtype=float32)
```

```
# con este tensor...
rank_2_tensor = tf.constant([[10, 7],
                             [3, 4]])

# muestra el último elemento de cada fila
print(rank_2_tensor[:, -1])

tf.Tensor([7 4], shape=(2,), dtype=int32)
```

También puedes añadir dimensiones a tu tensor mientras mantienes la misma información presente usando `tf.newaxis`.

```
# añade al tensor anterior una nueva dimensión (al final) con el método tf.newaxis
tensor_expandido = rank_2_tensor[..., tf.newaxis]
print("Shape original:", rank_2_tensor.shape)
print("Shape expandido:", tensor_expandido.shape)
```

```
Shape original: (2, 2)
Shape expandido: (2, 2, 1)
```

esto también se puede hacer con `tf.expand_dims()`

```
# expande un tensor de rango 2 en una dimensión por el último eje
tensor_expandido2 = tf.expand_dims(rank_2_tensor, axis=-1)
print("Shape con expand_dims:", tensor_expandido2.shape)
```

```
Shape con expand_dims: (2, 2, 1)
```

▼ `tf.gather`

El método `tf.gather` en TensorFlow extrae elementos específicos de un tensor a lo largo de un eje, según los índices especificados.

Ejemplo

```
import tensorflow as tf

# Crear un tensor de ejemplo
tensor = tf.constant([[1, 2], [3, 4], [5, 6]])

# Seleccionar filas con índices [0, 2]
gathered_tensor = tf.gather(tensor, [0, 2])

print(gathered_tensor.numpy()) # Resultado: [[1, 2], [5, 6]]
```

```
# de un tensor aleatorio con forma (10,4) selecciona las filas 3 y 7
tensor = tf.random.uniform((10, 4))
```

```
# Seleccionamos las filas 3 y 7
filas_seleccionadas = tf.gather(tensor, [3, 7])
print("Filas 3 y 7:\n", filas_seleccionadas)
```

```
Filas 3 y 7:
tf.Tensor(
[[0.13318717 0.5480639 0.5746088 0.8996835 ]
 [0.8527372 0.44062173 0.9485276 0.23752594]], shape=(2, 4), dtype=float32)
```

▼ Manipulando tensores (operaciones con tensores)

Encontrar patrones en tensores (representación numérica de los datos) requiere manipularlos.

De nuevo, cuando construyes modelos en TensorFlow, gran parte de este descubrimiento de patrones se hace por ti.

Operaciones básicas

Puedes realizar muchas de las operaciones matemáticas básicas directamente en tensores usando operadores de Python, tales como, `+`, `-`, `*`.

```
# utiliza las operaciones suma +, multiplicación * (tb con multiply) y resta - para crear
a = tf.constant([1, 2, 3])
b = tf.constant([10, 10, 10])

suma = a + b
producto = a * b
resta = a - b
producto2 = tf.multiply(a, b)

print("Suma:", suma.numpy())
print("Multiplicación:", producto.numpy())
print("Multiplicación con tf.multiply:", producto2.numpy())
print("Resta:", resta.numpy())
```

```
Suma: [11 12 13]
Multiplicación: [10 20 30]
Multiplicación con tf.multiply: [10 20 30]
Resta: [-9 -8 -7]
```

```
# comprueba que los tensores originales no se cambian cuando se utilizan estos operadores

print("a :", a.numpy())
print("b :", b.numpy())
```

```
a : [1 2 3]
b : [10 10 10]
```

▼ Multiplicación de matrices

Una de las operaciones más comunes en los algoritmos de aprendizaje automático es la [multiplicación de matrices](#).

TensorFlow implementa esta funcionalidad de multiplicación de matrices en el método `tf.matmul()`.

Las dos reglas principales para recordar sobre la multiplicación de matrices son:

1. Las dimensiones internas deben coincidir:

- `(3, 5) @ (3, 5)` no funcionará
- `(5, 3) @ (3, 5)` funcionará
- `(3, 5) @ (5, 3)` funcionará

2. La matriz resultante tiene la forma de las dimensiones exteriores:

- `(5, 3) @ (3, 5) -> (5, 5)`
- `(3, 5) @ (5, 3) -> (3, 3)`

🔑 **Nota:** '@' en Python es el símbolo para la multiplicación de matrices.

```
# utiliza la operación matmul para multiplicar dos tensores (tensor)
M1 = tf.constant([[1, 2],
                  [3, 4]])
M2 = tf.constant([[5, 6],
                  [7, 8]])

resultado_matmul = tf.matmul(M1, M2)
print("Resultado matmul:\n", resultado_matmul)
```

```
Resultado matmul:
tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int32)
```

```
# usa esta vez el operador @
resultado_arroba = M1 @ M2
print("Resultado con @:\n", resultado_arroba)
```

```
Resultado con @:
tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int32)
```

La multiplicación anterior funcionaba porque los dos eran tensores de 2,2, si creamos de dimensiones incompatibles:

```
# con estos tensores
# (3, 2)
X = tf.constant([[1, 2],
                 [3, 4],
                 [5, 6]])

# (3, 2)
Y = tf.constant([[7, 8],
                 [9, 10],
                 [11, 12]])

X, Y
```

```
(<tf.Tensor: shape=(3, 2), dtype=int32, numpy=
array([[1, 2],
       [3, 4],
       [5, 6]], dtype=int32)>,
 <tf.Tensor: shape=(3, 2), dtype=int32, numpy=
array([[ 7,  8],
       [ 9, 10],
       [11, 12]], dtype=int32)>)
```

```
# multiplícalos -> error!
try:
    resultado = X @ Y
except Exception as e:
    print("ERROR:", e)
```

```
ERROR: {{function_node __wrapped__MatMul_device_/job:localhost/replica:0/task:0/device:CP
```

Intentar multiplicar matrices de dos tensores con forma (3, 2) genera errores porque las dimensiones internas no coinciden.

Necesitamos:

- Cambiar la forma de X a $(2, 3)$ para que sea $(2, 3) @ (3, 2)$.
- Cambiar la forma de Y a $(3, 2)$ para que sea $(3, 2) @ (2, 3)$.

Podemos hacer esto con:

- `tf.reshape()` - nos permite cambiar la forma de un tensor a una forma definida.
- `tf.transpose()` - cambia las dimensiones de un tensor dado.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 & 64 \\ \end{bmatrix}$$

Probemos primero `tf.reshape()`.

```
#recuerda los valores de X e Y
X,Y
```

```
(<tf.Tensor: shape=(3, 2), dtype=int32, numpy=
array([[1, 2],
       [3, 4],
       [5, 6]], dtype=int32)>,
 <tf.Tensor: shape=(3, 2), dtype=int32, numpy=
array([[ 7,  8],
       [ 9, 10],
       [11, 12]], dtype=int32)>)
```

```
# cambia, con el método reshape la forma del segundo tensor a (2,3)
```

```
Y_reshape = tf.reshape(Y, shape=(2, 3))
print("Y con nueva forma:", Y_reshape.shape)
```

```
Y con nueva forma: (2, 3)
```

```
# prueba a multiplicar ahora
```

```
resultado_bien = X @ Y_reshape
print("Resultado:\n", resultado_bien)
```

```
Resultado:
tf.Tensor(
[[ 27  30  33]
 [ 61  68  75]
 [ 95 106 117]], shape=(3, 3), dtype=int32)
```

Funcionó, intentemos lo mismo con un `X` reconfigurado, excepto que esta vez usaremos

`tf.transpose()` y `tf.matmul()`.

```
# examina la transpuesta de X
X_transpuesta = tf.transpose(X)
print("X transpuesta:\n", X_transpuesta)
print("Shape:", X_transpuesta.shape)
```

```
X transpuesta:
tf.Tensor(
[[1 3 5]
```

```
[2 4 6]], shape=(2, 3), dtype=int32)
Shape: (2, 3)
```

```
# multiplica la transpuesta de X por Y
resultado_transpuesta = tf.matmul(tf.transpose(X), Y)
print("Resultado con transpuesta:\n", resultado_transpuesta)
```

```
Resultado con transpuesta:
tf.Tensor(
[[ 89  98]
 [116 128]], shape=(2, 2), dtype=int32)
```

Se puede obtener el mismo resultado con los `transpose_a` o `transpose_b`:

```
# multiplica los dos tensores anteriores usando los parámetros transpose
resultado_param = tf.matmul(X, Y, transpose_a=True)
print("Resultado con transpose_a=True:\n", resultado_param)
```

```
Resultado con transpose_a=True:
tf.Tensor(
[[ 89  98]
 [116 128]], shape=(2, 2), dtype=int32)
```

Observa la diferencia en las formas resultantes al transponer `X` o reconfigurar `Y`.

Esto se debe a la 2da regla mencionada anteriormente:

- $(3, 2) @ (2, 3) \rightarrow (3, 3)$ hecho con `X @ tf.reshape(Y, shape=(2, 3))`
- $(2, 3) @ (3, 2) \rightarrow (2, 2)$ hecho con `tf.matmul(tf.transpose(X), Y)`

Este tipo de manipulación de datos es un recordatorio: pasarás mucho de tu tiempo en el aprendizaje automático y trabajando con redes neuronales reconfigurando datos (en forma de tensores) para prepararlos para ser usados con varias operaciones (como alimentarlos a un modelo).

✓ El dot product (producto punto)

Multiplicar matrices de forma genérica también se conoce como el producto punto.

Puedes realizar la operación `tf.matmul()` usando `tf.tensordot()`.

`tf.tensordot()`:

Permite realizar el producto punto entre tensores, especificando los ejes sobre los cuales se quiere realizar la operación. Es más flexible ya que puedes especificar los ejes a lo largo de los cuales el producto punto debe ser calculado. Esto significa que `tf.tensordot()` puede ser usado no solo para la multiplicación de matrices, sino también para operaciones más generales de producto punto. Puede manejar tensores de cualquier rango, y los ejes a lo largo de los cuales se realiza el producto punto pueden ser configurados, permitiendo así una gama más amplia de operaciones de álgebra tensorial más allá de la simple multiplicación de matrices.

Las operaciones de producto punto (dot product) son fundamentales en álgebra lineal y tienen varias aplicaciones importantes, especialmente en el campo del aprendizaje automático y el procesamiento de señales. Aquí se describen varios tipos de operaciones de producto punto:

1. Producto Punto Escalar (Scalar Dot Product):

- Este es el caso más simple y se aplica a vectores de la misma dimensión. El resultado es un escalar. Por ejemplo, el producto punto de dos vectores $\mathbf{a}$ y $\mathbf{b}$ se calcula como:

$$\sum_{i=1}^n a_i b_i$$

2. Multiplicación de Matrices (Matrix Multiplication):

- Es una generalización del producto punto escalar a matrices. Si tienes dos matrices, A de tamaño

$$m \times n$$

y B de tamaño

$$n \times p$$

, el resultado es una nueva matriz C de tamaño

$$m \times p$$

, donde cada elemento

$$C_{ij}$$

es el producto punto de la fila i de A y la columna j de B:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$$

3. Producto Tensorial (Tensor Dot Product):

- Esta operación extiende el concepto de producto punto a tensores de orden superior. En el contexto de tensores, el producto punto puede realizarse a lo largo de uno o más ejes especificados. Por ejemplo, si tienes dos tensores tridimensionales, puedes calcular el producto punto a lo largo de un eje particular, combinando las dimensiones correspondientes de cada tensor.

4. Producto Punto Interno (Inner Product):

- Es similar al producto punto escalar, pero se usa en el contexto de espacios de dimensión superior. En matemáticas, el producto interior generaliza el concepto de producto punto a espacios vectoriales de dimensiones más altas.

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^n a_i b_i$$

5. Producto Punto Externo (Outer Product):

- A diferencia del producto punto que produce un escalar (en el caso de vectores) o una matriz (en el caso de matrices), el producto externo toma dos vectores y produce una matriz. Si tienes dos vectores $\mathbf{a}$ y $\mathbf{b}$, el producto externo es una matriz donde cada elemento es:

$$C_{ij} = a_i b_j$$

6. Convolution:

- Aunque no es un producto punto en el sentido tradicional, la convolución en el procesamiento de señales y en redes neuronales convolucionales se puede conceptualizar como una operación de producto punto entre filtros y segmentos de la señal o imagen, especialmente cuando se implementa mediante operaciones de matriz.

Cada uno de estos tipos de operaciones de producto punto tiene aplicaciones específicas y se elige en función del problema a resolver, la naturaleza de los datos y los objetivos del análisis o del modelo de aprendizaje automático.

```
# Realiza el producto X e Y con tensordot(requiere la traspuesta de X)
resultado_tensordot = tf.tensordot(tf.transpose(X), Y, axes=1)
print("Resultado tensordot:\n", resultado_tensordot)
```

```
Resultado tensordot:
tf.Tensor(
[[ 89  98]
 [116 128]], shape=(2, 2), dtype=int32)
```

✓ Cambiando el tipo de dato de un tensor

A veces querrás alterar el tipo de dato predeterminado de tu tensor.

Esto es común cuando quieres computar usando menos precisión (por ejemplo, números de punto flotante de 16 bits vs. números de punto flotante de 32 bits).

Computar con menos precisión es útil en dispositivos con menor capacidad de cómputo como dispositivos móviles (porque menos bits, requieren menos espacio en las computaciones).

Puedes cambiar el tipo de dato de un tensor usando `tf.cast()`.

```
# con estos tensores
B = tf.constant([1.7, 7.4])
C = tf.constant([1, 7])

# ¿Qué tipo tienen?
# intenta sumarlos
print("B dtype:", B.dtype) # float32
print("C dtype:", C.dtype) # int32

# Si se intentan sumar da error porque son tipos distintos
try:
    print(B + C)
except Exception as e:
    print("ERROR:", e)
```

```
B dtype: <dtype: 'float32'>
C dtype: <dtype: 'int32'>
ERROR: cannot compute AddV2 as input #1(zero-based) was expected to be a float tensor but
```

```
# convierte B float32 a float16 (precisión reducida)
B_float16 = tf.cast(B, dtype=tf.float16)
print("B en float16:", B_float16)
print("Dtype:", B_float16.dtype)
```

```
B en float16: tf.Tensor([1.7 7.4], shape=(2,), dtype=float16)
Dtype: <dtype: 'float16'>
```

```
# convierte C de int32 a float32
C_float32 = tf.cast(C, dtype=tf.float32)
print("C en float32:", C_float32)
print("Ahora sí podemos sumar:", B + C_float32)
```

```
C en float32: tf.Tensor([1. 7.], shape=(2,), dtype=float32)
Ahora sí podemos sumar: tf.Tensor([ 2.7 14.4], shape=(2,), dtype=float32)
```

✓ Obteniendo el valor absoluto

A veces querrás los valores absolutos (todos los valores son positivos) de los elementos en tus tensores.

Para hacerlo, puedes usar `tf.abs()`.

```
D = tf.constant([-7, -10])
# obten el valor absoluto de este tensor

print("Valor absoluto:", tf.abs(D).numpy())
```

```
Valor absoluto: [ 7 10]
```

✓ Encontrando el mínimo, máximo, media, suma (agregación)

Es posible agregar (realizar un cálculo en todo un tensor) tensores para encontrar cosas como el valor mínimo, valor máximo, media y suma de todos los elementos.

Para hacerlo, los métodos de agregación típicamente tienen la sintaxis `reduce()_acción`, tales como:

- `tf.reduce_min()` - encontrar el valor mínimo en un tensor.
- `tf.reduce_max()` - encontrar el valor máximo en un tensor (útil cuando quieres encontrar la probabilidad de predicción más alta).
- `tf.reduce_mean()` - encontrar la media de todos los elementos en un tensor.
- `tf.reduce_sum()` - encontrar la suma de todos los elementos en un tensor.
- **Nota:** típicamente, cada uno de estos está bajo el módulo `math`, por ejemplo, `tf.math.reduce_min()` pero puedes usar el alias `tf.reduce_min()`.

Veámoslos en acción.

```
# con este tensor con 50 valores random entre 0 y 100
E = tf.constant(np.random.randint(low=0, high=100, size=50))
E
```

```
<tf.Tensor: shape=(50,), dtype=int64, numpy=
array([81, 29,  7, 59, 73, 12, 26, 25, 52, 53,  8, 48, 23, 39, 70, 56, 86,
       94, 38, 98, 57, 22, 29, 84, 93, 44, 59, 36, 61, 96, 66, 34, 58, 85,
       57, 70, 74, 15, 16, 53,  5, 74, 21, 78,  2,  1, 57, 25, 55, 85])>
```

```
# calcula el mínimo
print("Mínimo de E:", tf.reduce_min(E).numpy())
```

```
Mínimo de E: 1
```

```
## calcula el máximo
print("Máximo de E:", tf.reduce_max(E).numpy())
```

```
Máximo de E: 98
```

```
# calcula la media
print("Media de E:", tf.reduce_mean(tf.cast(E, tf.float32)).numpy())
```

Media de E: 49.78

```
#calcula la suma
print("Suma de E:", tf.reduce_sum(E).numpy())
```

Suma de E: 2489

También puedes calcular la desviación estándar (`tf.reduce_std()`) y la varianza (`tf.reduce_variance()`) de los elementos en un tensor usando métodos similares.

✓ Encontrando el máximo y mínimo posicional

¿Y cómo se busca la posición en un tensor donde está el valor máximo?

Esto es útil cuando quieres alinear tus etiquetas (digamos `['Verde', 'Azul', 'Rojo']`) con tu tensor de probabilidades de predicción (por ejemplo, `[0.98, 0.01, 0.01]`).

En este caso, la etiqueta predicha (la que tiene la probabilidad de predicción más alta) sería `'Verde'`.

Puedes hacer lo mismo para el mínimo (si es necesario) con lo siguiente:

- `tf.argmax()` - encontrar la posición del elemento máximo en un tensor dado.
- `tf.argmin()` - encontrar la posición del elemento mínimo en un tensor dado.

```
# con este tensor con 50 valores entre 0 y 1
F = tf.constant(np.random.random(50))
F
```

```
<tf.Tensor: shape=(50,), dtype=float64, numpy=
array([0.94959403, 0.02043724, 0.30322459, 0.49544828, 0.36482394,
        0.06890884, 0.64617806, 0.37522014, 0.86831779, 0.79185726,
        0.87517355, 0.65299686, 0.91619836, 0.5068975 , 0.50757476,
        0.55134885, 0.69835141, 0.07421792, 0.02391848, 0.07242231,
        0.35520521, 0.24712766, 0.05492032, 0.00600087, 0.9645015 ,
        0.86964641, 0.72998473, 0.10778122, 0.40304501, 0.47691912,
        0.99492425, 0.68276666, 0.37336521, 0.2014666 , 0.0658243 ,
        0.77527638, 0.79088874, 0.14172034, 0.26321134, 0.12262078,
        0.6585669 , 0.73046136, 0.95752219, 0.11689773, 0.47579789,
        0.60637285, 0.51064368, 0.25122687, 0.30998599, 0.7341657 ])>
```

```
# buscar la posición del elemento máximo de F
pos_max = tf.argmax(F)
print("Posición del máximo:", pos_max.numpy())
print("Valor en esa posición:", F[pos_max].numpy())
```

Posición del máximo: 30
Valor en esa posición: 0.9949242486739525

```
# buscar la posición del elemento mínimo de F
pos_min = tf.argmin(F)
print("Posición del mínimo:", pos_min.numpy())
```

Posición del mínimo: 23

```
pos_min = tf.argmax(F)
```

```
# Encuentra el valor mínimo de F
valor_min = tf.reduce_min(F)
```

```
valor_min_indexado = F[pos_min]
# ¿Son los dos valores mínimos iguales? (deberían serlo!)
print("Posición del mínimo:", pos_min.numpy())
print("Mínimo con reduce_min:", valor_min.numpy())
print("Mínimo con argmin+índice:", valor_min_indexado.numpy())
print("¿Son iguales?", tf.equal(valor_min, valor_min_indexado).numpy())
```

```
Posición del mínimo: 23
Mínimo con reduce_min: 0.006000869780159679
Mínimo con argmin+índice: 0.006000869780159679
¿Son iguales? True
```

✓ Comprimiendo un tensor (eliminando todas las dimensiones unitarias)

Si necesitas eliminar las dimensiones unitarias de un tensor (dimensiones de tamaño 1), puedes usar `tf.squeeze()`.

- `tf.squeeze()` - elimina todas las dimensiones de 1 de un tensor.

```
# Con este tensor de rango 5 con números entre 0 y 100
G = tf.constant(np.random.randint(0, 100, 50), shape=(1, 1, 1, 1, 50))
G.shape, G.ndim
```

```
(TensorShape([1, 1, 1, 1, 50]), 5)
```

```
# elimina con tf.squeeze() todas las dimensiones 1
G_comprimido = tf.squeeze(G)
print("Shape original:", G.shape)
print("Shape tras squeeze:", G_comprimido.shape)
```

```
Shape original: (1, 1, 1, 1, 50)
Shape tras squeeze: (50,)
```

✓ Codificación one-hot

La **codificación one-hot** es una técnica para convertir categorías en vectores numéricos, donde cada categoría es representada por un vector de 0s y un único 1 en la posición correspondiente.

Ejemplo

Imagina tres categorías: ["rojo", "verde", "azul"].

- "rojo": `[1, 0, 0]`
- "verde": `[0, 1, 0]`
- "azul": `[0, 0, 1]`

Cada categoría tiene un vector con longitud igual al número total de categorías, y solo una posición en `1`.

¿Para qué sirve?

- Útil en machine learning para convertir etiquetas categóricas en una forma que los algoritmos puedan procesar.

Implementación en TensorFlow

```
import tensorflow as tf

categories = [0, 1, 2] # Ejemplo de índices para ["rojo", "verde", "azul"]
one_hot_encoded = tf.one_hot(categories, depth=3)

print(one_hot_encoded.numpy())
# Resultado:
# [[1. 0. 0.]
#  [0. 1. 0.]
#  [0. 0. 1.]]
```

Si tienes un tensor de índices y te gustaría codificarlo en one-hot, puedes usar `tf.one_hot()`. También deberías especificar el parámetro `depth` (el nivel al que quieres codificar en one-hot).

```
# Con esta lista de índices
some_list = [0, 1, 2, 3]

# codificarlos en One hot usa el parámetro depth=4
one_hot = tf.one_hot(some_list, depth=4)
print(one_hot)
```

```
tf.Tensor(
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]], shape=(4, 4), dtype=float32)
```

```
# Especifica valores personalizados para on y off
one_hot_custom = tf.one_hot(some_list, depth=4, on_value=33.0, off_value=-1.0)
print(one_hot_custom)
```

```
tf.Tensor(
[[33. -1. -1. -1.]
 [-1. 33. -1. -1.]
 [-1. -1. 33. -1.]
 [-1. -1. -1. 33.]], shape=(4, 4), dtype=float32)
```

▼ Operaciones matemáticas (log, raíz cuadrada, cuadrado...)

Tensorflow también ofrece operaciones matemáticas como:

- `tf.square()` - cuadrado de todos los valores de un tensor.
- `tf.sqrt()` - raíz cuadrada de todos los elementos de un tensor
- `tf.math.log()` - logaritmo de todos los elementos (base 10)

```
# con este tensor
H = tf.constant(np.arange(1, 10))

# imprime su cuadrado con square
print("Cuadrado:", tf.square(H).numpy())

Cuadrado: [ 1  4  9 16 25 36 49 64 81]
```

```
# imprime su raíz cuadrada
```

```
# ¿te ha dado error? (ojo, no debe ser entero)
try:
    print(tf.sqrt(H))
except Exception as e:
    print("ERROR:", e)
```

```
ERROR: Value for attr 'T' of int64 is not in the list of allowed values: bfloat16, half,
; NodeDef: {{node Sqrt}}; Op<name=Sqrt; signature=x:T -> y:T; attr=T:type,allowed
```

```
# hazle un cast a float32 y repite la raíz cuadrada
H_float = tf.cast(H, dtype=tf.float32)
print("Raíz cuadrada:", tf.sqrt(H_float).numpy())
```

```
Raíz cuadrada: [1.          1.4142135 1.7320508 2.          2.236068  2.4494898 2.6457512
 2.828427  3.          ]
```

```
# saca el logaritmo del tensor
print("Logaritmo:", tf.math.log(H_float).numpy())
```

```
Logaritmo: [0.          0.6931472 1.0986123 1.3862944 1.609438  1.7917595 1.9459102
 2.0794415 2.1972246]
```

▼ Tensores y NumPy

Hemos visto algunos ejemplos de cómo los tensores interactúan con los arrays de NumPy, por ejemplo, utilizando arrays de NumPy para crear tensores.

Los tensores también pueden ser convertidos a arrays de NumPy utilizando:

- `np.array()` - pasa un tensor para convertirlo en un ndarray (el tipo de dato principal de NumPy).
- `tensor.numpy()` - se llama en un tensor para convertirlo en un ndarray.

Hacer esto es útil ya que hace que los tensores sean iterables, así como también nos permite usar cualquiera de los métodos de NumPy en ellos.

```
#crear un tensor desde un array de numpy
J = tf.constant(np.array([3., 7., 10.]))
J
```

```
<tf.Tensor: shape=(3,), dtype=float64, numpy=array([ 3.,  7., 10.])>
```

```
# Numpy puede convertir tensores en numpy arrays
np.array(J, type(np.array(J)))

(array([ 3.,  7., 10.]), numpy.ndarray)
```

```
# Convierte el tensor J a NumPy con .numpy()
J_numpy = J.numpy()
print("J como numpy:", J_numpy)
print("Tipo:", type(J_numpy))
```

```
J como numpy: [ 3.  7. 10.]
Tipo: <class 'numpy.ndarray'>
```

```
# Crea un tensor desde NumPy y desde un array de python y examina los tipos de datos que

desde_numpy = tf.constant(np.array([1.0, 2.0, 3.0]))
print("Desde numpy, dtype:", desde_numpy.dtype)

# Desde lista
desde_lista = tf.constant([1.0, 2.0, 3.0])
print("Desde lista Python, dtype:", desde_lista.dtype)

Desde numpy, dtype: <dtype: 'float64'>
Desde lista Python, dtype: <dtype: 'float32'>
```

✓ Usando `@tf.function`

En tus aventuras con TensorFlow, podrías encontrarte con funciones de Python que tienen el decorador `@tf.function`.

Si no estás seguro de qué hacen los decoradores de Python, [lee la guía de RealPython sobre ellos](#).

Pero en resumen, los decoradores modifican una función de una manera u otra.

En el caso del decorador `@tf.function`, convierte una función de Python en un grafo de TensorFlow llamable. Que es una forma elegante de decir, si has escrito tu propia función de Python, y la decoras con `@tf.function`, cuando exportas tu código (para potencialmente ejecutar en otro dispositivo), TensorFlow intentará convertirlo en una versión más rápida de sí mismo (haciéndola parte de un grafo de computación).

Para más información sobre esto, lee la guía [Mejor rendimiento con tf.function](#).

```
# crear una función sencilla
def function(x, y):
    return x ** 2 + y

x = tf.constant(np.arange(0, 10))
y = tf.constant(np.arange(10, 20))
function(x, y)
```

```
<tf.Tensor: shape=(10,), dtype=int64, numpy=array([ 10, 12, 16, 22, 30, 40, 52,
66, 82, 100])>
```

```
# Crear la misma función con el decorador tf.function
@tf.function
def tf_function(x, y):
    return x ** 2 + y

tf_function(x, y)
```

```
<tf.Tensor: shape=(10,), dtype=int64, numpy=array([ 10, 12, 16, 22, 30, 40, 52,
66, 82, 100])>
```

Verás que no hay diferencia visible entre las dos funciones anterior. Las diferencias están en la forma de ejecutarse

Verás que no hay diferencia visible entre las dos funciones anterior. Las diferencias están en la forma de ejecutarse

✓ Encontrando acceso a GPUs

¿cómo se puede verificar si hay una disponible?

Puedes comprobar si tienes acceso a una GPU utilizando

```
tf.config.list_physical_devices().
```

```
# imprime los dispositivos GPU de los que dispones
gpus = tf.config.list_physical_devices("GPU")
print("GPUs disponibles:", gpus)
```

```
GPUs disponibles: []
```

- En este caso no muestra ninguna

```
# si tienes linux, puedes ejecutar el comando nvidia-smi
#nvidia-smi
```

🔑 **Nota:** Si tienes acceso a una GPU, automáticamente Tensorflow la usará cuando sea posible